

NETPROBE: UDP TABANLI GÜVENİLİR DOSYA AKTARIMI, TRAFİK İZLEME VE AĞ PERFORMANS ANALIZI

Ders: Bilgisayar Ağları
Proje Türü: Dönem Projesi
Proje Başlığı: NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı, Trafik İzleme ve Ağ Performans Analiz Platformu
Hazırlayan: Rima Farah Eleuch
GitHub Deposu: <https://github.com/Rima2002/netprobe>
Tarih: Haziran 2026

ÖZET

Bu teknik rapor, NetProbe adlı UDP tabanlı güvenilir dosya aktarım projesinin tasarımını, gerçekleştirme ayrıntılarını, deney ortamını, performans metriklerini ve deney sonuçlarını açıklamaktadır. Projenin temel amacı, UDP protokolünün doğasında bulunmayan güvenilir veri aktarımı özelliklerini uygulama katmanında elle tasarlamak ve geliştirmektir. Bu kapsamda istemci-sunucu mimarisi kurulmuş; dosya parçalama, sequence number, ACK, timeout, retransmission, duplicate packet kontrolü ve SHA-256 bütünlük doğrulaması uygulanmıştır.

Proje yalnızca dosya aktarımı yapmakla kalmaz; aktarım sırasında oluşan olayları CSV/JSON formatında loglar, bu loglardan throughput, goodput, RTT, completion time, packet loss rate ve retransmission rate gibi metrikleri hesaplar. Ayrıca farklı paket boyutu, timeout değeri ve yapay kayıp oranı senaryoları için deneyler çalıştırır ve sonuçları grafiklerle destekler. Raporun sonunda deney sonuçları protokol davranışıyla ilişkilendirilerek teknik olarak yorumlanmıştır.

İÇİNDEKİLER

1. Giriş
2. Problem Tanımı
3. Sistem Mimarisi
4. Protokol Tasarımı
5. Gerçekleme Detayları
6. Deney Ortamı
7. Performans Metrikleri
8. Deney Sonuçları ve Tartışma
9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları
10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler
11. Teslim Edilen Çıktılar
12. Kaynaklar ve Kullanılan Kütüphaneler

1. GİRİŞ

Bilgisayar ağlarında güvenilir veri aktarımı, özellikle dosya gönderimi gibi veri bütünlüğünün önemli olduğu uygulamalarda temel bir gereksinimdir. TCP protokolü güvenilir aktarım, sıralama, akış kontrolü ve yeniden gönderim gibi mekanizmaları kendi içinde sağlar. Buna karşılık UDP daha basit, bağlantısız ve düşük ek yüke sahip bir taşıma katmanı protokolüdür. UDP paketlerin hedefe ulaşacağını, doğru sırada geleceğini veya kaybolan paketlerin yeniden gönderileceğini garanti etmez.

Bu proje, UDP üzerinde güvenilir dosya aktarımı yapılabilmesi için gerekli mekanizmaların uygulama katmanında nasıl tasarlanabileceğini göstermeyi amaçlar. NetProbe, istemciden sunucuya dosya gönderen ve bu aktarımı Stop-and-Wait ARQ yaklaşımı ile güvenilir hale getiren bir sistemdir. Her veri paketi sequence number ile numaralandırılır, sunucu doğru aldığı paketler için ACK üretir, istemci ACK gelmezse timeout sonrasında aynı paketi yeniden gönderir. Sunucu duplicate paket aldığı anda aynı veriyi ikinci kez dosyaya yazmaz; yalnızca ilgili ACK'i tekrar gönderir.

Proje ayrıca ağ performansını ölçmek için loglama ve analiz bileşenleri içerir. Böylece yalnızca çalışan bir aktarım sistemi değil, aynı zamanda bu sistemin farklı ağ koşullarındaki davranışını inceleyen küçük bir performans analiz platformu oluşturulmuştur.

2. PROBLEM TANIMI

UDP protokolü hızlı ve düşük ek yüklü olmasına rağmen güvenilir dosya aktarımı için tek başına yeterli değildir. UDP ile gönderilen bir datagram ağda kaybolabilir, bozulabilir, hedefe geç ulaşabilir veya uygulama tarafından beklenen sırada alınmayabilir. Dosya aktarımında bu durumlar ciddi sorunlara yol açar; çünkü tek bir eksik veya bozuk parça bile dosyanın tamamını kullanılamaz hale getirebilir.

Bu projenin çözmesi gereken temel problemler şunlardır:

- Dosyanın UDP datagramlarına sığacak parçalara bölünmesi.
- Her parçanın sıra numarası ile takip edilmesi.
- Alıcının doğru aldığı paketler için ACK üretmesi.
- Göndericinin ACK gelmediğinde timeout algılaması.
- Kaybolan paketlerin yeniden gönderilmesi.
- Duplicate paketlerin ikinci kez dosyaya yazılmaması.
- Dosya parçalarının doğru sırada yeniden birleştirilmesi.
- Aktarım sonunda dosya bütünlüğünün doğrulanması.
- Aktarım olaylarının loglanması ve performans metriklerinin hesaplanması.

Projenin zorunlu gereksinimleri doğrultusunda hazır bir dosya aktarım kütüphanesi kullanılmamıştır. Dosya aktarımı doğrudan Python socket modülü ile UDP üzerinden gerçekleştirilmiştir. Güvenilirlik mekanizması TCP'den hazır alınmamış, uygulama katmanında elle tasarlanmıştır.

3. SİSTEM MIMARISI

NetProbe modüler bir Python projesi olarak tasarlanmıştır. Her dosya belirli bir sorumluluğa sahiptir:

```
| Dosya/Klasör | Görev |
| --- | --- |
| client.py | UDP istemcisi; dosyayı parçalar, paketleri Stop-and-Wait ARQ ile gönderir, ACK bekler ve log üretir. |
| server.py | UDP sunucusu; START/DATA/FIN paketlerini işler, parçaları saklar, dosyayı yeniden oluşturur ve ACK gönderir. |
| protocol.py | Paket formatı, paket oluşturma/çözme, payload checksum ve SHA-256 dosya hash fonksiyonlarını içerir. |
| logger.py | Aktarım olaylarını CSV ve JSON olarak kaydeder. |
| analyzer.py | Log dosyalarından performans metrikleri, grafikler ve teknik yorum dosyası üretir. |
| experiments.py | Otomatik deney senaryolarını çalıştırır ve sonuçları CSV/JSON/grafik olarak kaydeder. |
| tcp_client.py, tcp_server.py | Bonus olarak UDP/TCP karşılaştırması için kullanılan yardımcı betiklerdir. |
| tests/ | Unit test dosyalarını içerir. |
| test_files/ | Deney ve demo için kullanılan test dosyalarını içerir. |
| logs/ | İstemci ve sunucu transfer loglarının üretildiği klasördür. |
| results/ | Analiz özetleri, deney sonuçları, grafikler ve teknik yorum çıktılarının üretildiği klasördür. |
```

Sistemin çalışma akışı şu şekildedir:

1. Kullanıcı sunucuyu başlatır.
2. Kullanıcı istemciye gönderilecek dosyayı verir.
3. İstemci dosyayı sabit boyutlu parçalara böler.
4. İstemci START paketiyle dosya metadeta bilgisini sunucuya yollar.
5. Sunucu START paketini ACK ile onaylar.
6. İstemci her DATA paketini gönderir ve aynı paket için ACK bekler.
7. ACK gelmezse istemci timeout sonrası aynı sequence number değerine sahip paketi yeniden gönderir.
8. Tüm DATA paketleri tamamlandığında istemci FIN paketi gönderir.
9. Sunucu dosyayı yeniden oluşturur, SHA-256 kontrolü yapar ve FIN_ACK döndürür.
10. İstemci ve sunucu loglarını kaydeder.
11. Analyzer logları okuyarak metrikleri ve grafikleri üretir.

4. PROTOKOL TASARIMI

NetProbe kendi uygulama katmanı paket formatını kullanır. Paket başlığı protocol.py içinde struct modülü ile tanımlanmıştır:

```
text
!BIIH32s
```

Bu format aşağıdaki alanlardan oluşur:

Alan Boyut Açıklama
--- --- ---
packet_type 1 byte Paketin türünü belirtir.
sequence_number 4 byte DATA paketleri için sıra numarasıdır.
total_packets 4 byte Transferdeki toplam DATA paketi sayısıdır.
payload_length 2 byte Payload uzunluğunu belirtir.
checksum 32 byte Payload için SHA-256 özeti.
payload Değişken Dosya parçası veya kontrol bilgisi.

Kullanılan paket türleri:

Paket Türü Görevi
--- ---
START Dosya adı, toplam paket sayısı, dosya boyutu ve beklenen SHA-256 hash değerini taşır.
DATA Dosyanın bir parçasını taşır.
ACK START veya DATA paketinin başarıyla alındığını bildirir.
FIN İstemcinin veri gönderimini bitirdiğini bildirir.
FIN_ACK Sunucunun dosyayı yeniden oluşturduğunu ve bütünlük sonucunu bildirir.
ERROR START olmadan DATA/FIN gelmesi gibi hatalı durumları bildirir.

4.1. Stop-and-Wait ARQ

Projenin ana güvenilirlik mekanizması Stop-and-Wait ARQ'dur. Bu yaklaşımda istemci aynı anda yalnızca bir DATA paketi gönderir. Bir sonraki pakete geçebilmek için gönderilen paketin ACK mesajını bekler.

Stop-and-Wait akışı:

- 1. İstemci sequence number n olan DATA paketini gönderir.
- 2. Sunucu paketi alır, checksum doğrulaması yapar.
- 3. Paket geçerliyse sunucu parçayı saklar ve ACK gönderir.
- 4. İstemci doğru ACK'i alırsa sequence number n+1 olan pakete geçer.
- 5. İstemci timeout süresi içinde ACK alamazsa aynı paketi yeniden gönderir.
- 6. Aynı paket en fazla 5 kez yeniden gönderilir.
- 7. Maksimum deneme sayısı aşılsa aktarım başarısız kabul edilir.

Bu tasarım TCP kadar verimli değildir; çünkü her paket için bekleme vardır. Ancak proje gereksinimleri açısından anlaşılır, test edilebilir ve güvenilir aktarım mantığını açık biçimde gösterir.

4.2. Duplicate Packet Handling

ACK paketleri de ağda kaybolabileceği için sunucu daha önce aldığı bir DATA paketini tekrar alabilir. Böyle bir durumda sunucu aynı payload'u ikinci kez dosyaya yazmaz. Bunun yerine ilgili sequence number için ACK'i tekrar gönderir. Böylece hem veri bütünlüğü korunur hem de istemcinin beklediği ACK yeniden sağlanır.

4.3. Bütünlük Kontrolü

Her paket payload alanı için SHA-256 tabanlı checksum tutulur. Böylece bozuk payload içeren paketler tespit edilebilir. Aktarımın sonunda ise bütün dosyanın SHA-256 hash değeri karşılaştırılır. İstemci START/FIN aşamalarında beklenen hash bilgisini gönderir; sunucu dosyayı yeniden oluşturduktan sonra kendi hesapladığı hash ile karşılaştırır. Sonuç integrity_ok=true veya false olarak loglara yazılır.

5. GERÇEKLEME DETAYLARI

5.1. İstemci Gerçeklemesi

İstemci kodu client.py dosyasındadır. Kod final teslim için sadeleştirilmiş ve ana akış açık hale getirilmiştir:

- 1. Dosya yolu kontrol edilir.
- 2. Dosyanın SHA-256 hash değeri hesaplanır.

3. Dosya split_file fonksiyonu ile parçalara bölünür.
4. UDP socket oluşturulur.
5. START paketi güvenilir şekilde gönderilir.
6. DATA paketleri Stop-and-Wait ARQ ile sırayla gönderilir.
7. FIN paketi gönderilir.
8. Sunucudan gelen FIN_ACK içindeki integrity_ok sonucu okunur.
9. CSV/JSON log dosyaları kaydedilir.

İstemcideki en önemli fonksiyon send_reliable_packet fonksiyonudur. Bu fonksiyon bir paketi gönderir, doğru ACK gelene kadar bekler ve timeout oluşursa aynı paketi yeniden gönderir. Böylece START, DATA ve FIN paketleri aynı güvenilir gönderim mantığını kullanır.

5.2. Sunucu Gerçeklemesi

Sunucu kodu server.py dosyasındadır. Sunucu UDP socket ile belirtilen IP ve port üzerinde dinler. Gelen paketler önce parse_packet ile çözülür, ardından verify_packet ile checksum doğrulaması yapılır.

Sunucu tarafında her istemci için bir TransferSession yapısı tutulur. Bu yapı şu bilgileri içerir:

- istemci adresi
- dosya adı
- toplam paket sayısı
- beklenen SHA-256 değeri
- beklenen dosya boyutu
- alınan chunk sözlüğü
- duplicate packet sayısı
- bozuk paket sayısı

DATA paketleri sequence number değerine göre sözlükte saklanır. FIN paketi geldiğinde eksik sequence number var mı kontrol edilir. Eksik yoksa dosya doğru sırada yeniden oluşturulur ve SHA-256 kontrolü yapılır.

5.3. Loglama

logger.py içinde bulunan EventLogger, aktarım olaylarını hem CSV hem JSON olarak kaydeder. Loglanan başlıca olaylar şunlardır:

- transfer_started
- packet_sent
- ack_received
- timeout
- simulated_packet_loss
- packet_failed
- packet_received
- chunk_stored
- duplicate_packet_ignored
- file_reconstructed
- transfer_completed
- transfer_finished

Bu loglar analyzer tarafından okunarak performans metrikleri hesaplanır.

5.4. Analiz ve Grafik Üretimi

analyzer.py, log dosyalarını okuyarak metrik üretir. Ayrıca deney sonuçlarından grafikler oluşturur ve rapora eklenebilecek technical_interpretation.txt dosyasını üretir.

Üretilen temel çıktı dosyaları:

- results/analysis_summary.csv
- results/analysis_summary.json
- results/experiment_results.csv
- results/experiment_results.json
- results/technical_interpretation.txt
- results/packet_size_results.png
- results/timeout_results.png
- results/loss_rate_results.png

5.5. Unit Testler

Projenin doğruluğunu desteklemek için tests/ klasörü eklenmiştir. Testler python -m unittest discover tests komutu ile çalıştırılmıştır. Toplam 7 test başarıyla geçmiştir.

Test edilen konular:

- checksum hesaplama
- DATA packet oluşturma ve parse etme
- ACK packet oluşturma ve parse etme
- aynı dosya için SHA-256 hash değerinin tutarlı olması
- duplicate packet'in ikinci kez saklanmaması
- analyzer metriklerinin beklenen değerleri hesaplaması

6. DENEY ORTAMI

Deneyler yerel geliştirme ortamında, loopback IP adresi kullanılarak çalıştırılmıştır.

Bileşen	Değer	
---	---	
İşletim Sistemi	Windows	
Programlama Dili	Python 3.12	
Aktarım Ortamı	127.0.0.1 loopback	
Ana UDP Portu	5005	
Deney Portları	6100 ve sonrası	
Paket Boyutu Varsayılana	1024 byte	
Timeout Varsayılana	1.0 saniye	
Maksimum Yeniden Gönderim	5	
Harici Kütüphaneler	pandas, matplotlib	

Kullanılan temel komutlar:

```
cmd
python -m pip install -r requirements.txt
python -m compileall .
python -m unittest discover tests
python server.py --host 127.0.0.1 --port 5005 --once
python client.py --server-ip 127.0.0.1 --server-port 5005 --file test_files/medium.bin
python analyzer.py --log logs/sample_client_log.csv
python experiments.py
```

Test dosyaları:

Dosya	Boyut	Kullanım Amacı	
---	---	---	
small.txt	64 KB	Timeout ve loss rate deneyleri	
medium.bin	1 MB	Demo aktarımı, paket boyutu ve protokol karşılaştırması	
large.bin	10 MB	Büyük dosya aktarımı için hazır test verisi	

7. PERFORMANS METRIKLERİ

Projede aşağıdaki metrikler hesaplanmıştır:

7.1. Throughput

Throughput, aktarım sırasında gönderilmeye çalışılan toplam DATA payload miktarının aktarım süresine bölünmesiyle hesaplanır. Bu metrik retransmission ve yapay kayıp denemelerini de kapsar.

```
text
throughput = gönderilmeye çalışılan DATA byte / completion_time
```

7.2. Goodput

Goodput, alıcıda başarıyla yeniden oluşturulan gerçek dosya byte miktarının aktarım süresine bölünmesiyle hesaplanır. Goodput, kullanıcıya gerçekten faydalı olan veri hızını temsil eder.

text
goodput = orijinal dosya byte / completion_time

7.3. Packet Loss Rate

Yapay olarak kaybettirilen paket denemelerinin toplam gönderim denemelerine oranıdır.

text
packet_loss_rate = simulated_packet_loss / attempted_sends

7.4. Retransmission Count ve Retransmission Rate

Retransmission count, ilk denemeden sonra tekrar gönderilen DATA paketlerinin sayısıdır. Retransmission rate ise bu sayının toplam gönderim denemelerine oranıdır.

7.5. Average RTT

ACK alınan paketler için gönderim zamanı ile ACK alınma zamanı arasındaki farkların ortalamasıdır.

7.6. Completion Time

Transferin başlangıcı ile tamamlanması arasındaki toplam süredir.

7.7. Integrity Status

Aktarım sonunda sunucuda yeniden oluşturulan dosyanın SHA-256 hash değeri ile istemcinin gönderdiği beklenen hash değerinin eşleşip eşleşmediğini gösterir.

8. DENEY SONUÇLARI VE TARTIŞMA

8.1. Örnek Başarılı Aktarım

medium.bin dosyası için yapılan örnek aktarımda aşağıdaki sonuçlar alınmıştır:

Metrik	Değer
Dosya boyutu	1,048,576 byte
Başarılı DATA paketi	1024
Başarısız paket	0
Timeout sayısı	0
Retransmission count	0
Packet loss rate	0.0
Completion time	0.228082 saniye
Throughput	4,597,364.11 B/s
Goodput	4,597,364.11 B/s
Average RTT	0.0002018 saniye
Integrity	true

Bu sonuç, kayıpsız yerel ortamda sistemin dosyayı eksiksiz aktardığını göstermektedir. Orijinal test_files/medium.bin dosyası ile received_files/medium.bin dosyasının SHA-256 hash değerleri eşleşmiştir.

8.2. Paket Boyutunun Etkisi

Bu deneyde aynı medium.bin dosyası farklı paket boyutları ile aktarılmıştır. Timeout 1.0 saniye, loss rate 0.0 olarak sabit tutulmuştur.

	Paket Boyutu	Throughput (B/s)	Goodput (B/s)	Completion Time (s)	Retransmission
	---: ---: ---: ---: ---:				
	512	2,543,883.36	2,543,883.36	0.4122	0
	1024	4,311,969.01	4,311,969.01	0.2432	0
	2048	7,649,372.63	7,649,372.63	0.1371	0

Paket boyutu 512 bayttan 2048 bayta çıktığında goodput yaklaşık %200.70 artmıştır. Bunun temel nedeni Stop-and-Wait ARQ mekanizmasında her paket için ayrı ACK beklenmesidir. Paket boyutu büyüdükçe aynı dosya daha az sayıda DATA paketi ile gönderilir. Böylece ACK bekleme turu sayısı azalır, protokol ek yükü düşer ve completion time kısalır.

Ancak bu sonuç loopback ortamında ve kayıpsız koşulda elde edilmiştir. Gerçek ağlarda çok büyük paketler kaybolduğunda yeniden gönderim maliyeti daha yüksek olabilir. Bu nedenle paket boyutu seçimi ağ koşullarına göre dengelenmelidir.

İlgili grafik: results/packet_size_results.png

8.3. Timeout Değerinin Etkisi

Bu deneyde small.txt dosyası 0.1 yapay kayıp oranı ile aktarılmış, timeout değeri değiştirilmiştir.

	Timeout (s)	Throughput (B/s)	Goodput (B/s)	Completion Time (s)	Retransmission Count	Retransmission Rate
	---: ---: ---: ---: ---: ---:					
	0.2	43,856.55	38,983.60	1.6811	8	0.1111
	0.5	27,375.86	25,392.10	2.5810	5	0.0725
	1.0	22,127.18	21,136.41	3.1006	3	0.0448

Timeout 0.20 saniyeden 1.00 saniyeye çıktığında retransmission count 8'den 3'e düşmüştür. Daha uzun timeout, bazı paketlerin tekrar gönderilmeden önce daha uzun süre beklenmesine neden olur. Bu nedenle gereksiz retransmission azalabilir. Buna karşılık gerçek kayıp oluştuğunda istemci daha uzun beklediği için completion time artabilir. Deneyde timeout büyüdükçe completion time da artmıştır.

Bu sonuç, timeout değerinin çok küçük veya çok büyük seçilmemesi gerektiğini gösterir. Çok küçük timeout gereksiz yeniden gönderimlere, çok büyük timeout ise gerçek kayıplarda uzun bekleme sürelerine yol açabilir.

İlgili grafik: results/timeout_results.png

8.4. Yapay Paket Kaybının Etkisi

Bu deneyde small.txt dosyası 1024 byte paket boyutu ve 0.2 saniye timeout ile aktarılmıştır. Yapay kayıp oranı değiştirilmiştir.

	Loss Rate	Throughput (B/s)	Goodput (B/s)	Completion Time (s)	Retransmission Count	Retransmission Rate
	---: ---: ---: ---: ---: ---:					
	0.0	1,984,976.98	1,984,976.98	0.0330	0	0.0000
	0.1	47,551.07	43,475.26	1.5074	6	0.0857
	0.2	32,600.11	27,818.76	2.3558	11	0.1467

Kayıp oranı arttıkça completion time belirgin biçimde artmış, goodput ise düşmüştür. Loss rate 0.0 iken goodput yaklaşık 1,984,976.98 B/s iken loss rate 0.2 olduğunda 27,818.76 B/s seviyesine düşmüştür. Bunun nedeni istemcinin kaybolan paketler için timeout beklemesi ve aynı sequence number değerine sahip paketi yeniden göndermesidir.

Yapay kayıp oranı arttıkça retransmission rate de artmıştır. Bu durum Stop-and-Wait ARQ mekanizmasının güvenilirlik sağladığını, ancak kayıplı ortamda bekleme ve yeniden gönderim maliyeti nedeniyle performansın düştüğünü göstermektedir.

İlgili grafik: results/loss_rate_results.png

8.5. UDP/TCP Karşılaştırması

Bonus deney olarak aynı medium.bin dosyası UDP Stop-and-Wait ve TCP ile aktarılmıştır.

Protokol	Throughput (B/s)	Goodput (B/s)	Completion Time (s)
---	---	---	---
UDP Stop-and-Wait	3,952,565.12	3,952,565.12	0.2653
TCP	17,319,811.04	17,319,811.04	0.0605

TCP aktarımı loopback ortamında daha yüksek throughput üretmiştir. Bunun nedeni TCP'nin işletim sistemi tarafından optimize edilmiş olması ve Stop-and-Wait gibi her paket için uygulama seviyesinde bekleme yapmamasıdır. NetProbe'un amacı TCP'yi performans olarak geçmek değil, UDP üzerinde güvenilirlik mekanizmasının nasıl kurulabileceğini göstermektir. Bu nedenle TCP karşılaştırması, tasarlanan UDP mekanizmasının eğitim amaçlı niteliğini daha net ortaya koymaktadır.

8.6. Bütünlük Sonuçları

Tüm deney satırlarında integrity_ok=True elde edilmiştir. Bu sonuç, kayıp ve yeniden gönderim senaryolarına rağmen dosyaların sunucu tarafında eksiksiz ve doğru sırada yeniden oluşturulduğunu göstermektedir.

9. KARŞILAŞILAN SORUNLAR VE ÇÖZÜM YAKLAŞIMLARI

9.1. UDP'nin Güvenilir Olmaması

UDP paket kaybı, ACK kaybı ve sıralama garantisi vermediği için dosya aktarımı doğrudan yapılamaz. Bu sorun sequence number, ACK, timeout ve retransmission mekanizmaları ile çözülmüştür.

9.2. Duplicate Paketlerin Dosyayı Bozma Riski

ACK kaybolduğunda istemci aynı DATA paketini yeniden gönderebilir. Sunucu bu paketi ikinci kez yazarsa dosya bozulabilir. Bu nedenle sunucuda alınan sequence number değerleri sözlük içinde takip edilmiştir. Aynı sequence number tekrar gelirse veri tekrar yazılmamış, yalnızca ACK yeniden gönderilmiştir.

9.3. Timeout Değerinin Performansa Etkisi

Timeout küçük seçilirse gereksiz retransmission artabilir; büyük seçilirse gerçek kayıplarda bekleme süresi artar. Bu nedenle timeout değeri deneysel olarak incelenmiş ve raporda etkisi yorumlanmıştır.

9.4. Kod Karmaşıklığı

İlk sürümde istemci kodunda opsiyonel window size desteği bulunuyordu. Proje raporunda ana mekanizma Stop-and-Wait olarak anlatıldığı için bu durum karışıklık oluşturabilirdi. Final teslim öncesinde client.py sadeleştirilmiş, window size karmaşıklığı kaldırılmış ve sistem yalnızca Stop-and-Wait ARQ kullanacak şekilde netleştirilmiştir.

9.5. Sadece Grafik Üretmenin Yeterli Olmaması

Proje föyünde grafik üretmenin tek başına yeterli olmadığı, deney sonuçlarının teknik olarak yorumlanması gerektiği belirtilmiştir. Bu nedenle analyzer.py ve experiments.py çıktısına technical_interpretation.txt dosyası eklenmiştir. Bu dosya paket boyutu, timeout ve kayıp oranı etkilerini teknik olarak açıklar.

9.6. Teslim Klasörünün Düzenlenmesi

Çok sayıda timestamp'li log ve tekrar eden grafik dosyası teslim klasörünü karmaşık hale getiriyordu. Final teslim öncesinde klasör yapısı sadeleştirilmiş; örnek loglar, deney sonuçları ve gerekli grafikler ayrı klasörlerde tutulmuştur. .gitignore ile logs/, results/, received_files/ içindeki gereksiz çalışma çıktılarının GitHub'a yüklenmesi engellenmiştir.

10. SONUÇ VE GELECEKTE YAPILABİLECEK GELİŞTİRMELER

Bu projede UDP üzerinde güvenilir dosya aktarımı sağlayan bir uygulama katmanı protokolü geliştirilmiştir. Sistem istemci-sunucu mimarisi ile çalışmakta, dosyayı parçalara bölmekte, sequence number ve ACK kullanmakta, timeout durumunda yeniden gönderim yapmakta ve duplicate paketleri doğru şekilde ele almaktadır. Aktarım sonunda SHA-256 ile dosya bütünlüğü doğrulanmaktadır.

Yapılan deneyler, Stop-and-Wait ARQ mekanizmasının güvenilirlik sağladığını; ancak paket boyutu, timeout ve kayıp oranı gibi parametrelerin performansı ciddi şekilde etkilediğini göstermiştir. Paket boyutu büyüdükçe kayıpsız ortamda throughput artmış, yapay kayıp oranı arttıkça retransmission rate yükselmiş ve goodput düşmüştür. Timeout değerinin ise gereksiz retransmission ile bekleme süresi arasında denge kurması gerektiği görülmüştür.

Gelecekte yapılabilecek geliştirmeler:

- Stop-and-Wait yerine Sliding Window yaklaşımı eklenebilir.
- Selective Repeat veya Go-Back-N gibi daha gelişmiş ARQ mekanizmaları uygulanabilir.
- Gerçek ağ ortamında, farklı cihazlar arasında deneyler yapılabilir.
- Gecikme simülasyonu daha ayrıntılı hale getirilebilir.
- Basit bir gerçek zamanlı izleme paneli eklenebilir.
- Wireshark/pcap çıktılarıyla karşılaştırmalı analiz yapılabilir.
- Çoklu istemci desteği geliştirilebilir.
- Dosya sıkıştırma veya basit şifreleme desteği eklenebilir.

Sonuç olarak NetProbe, dönem projesi föyünde istenen UDP tabanlı güvenilir dosya aktarımı, trafik izleme ve performans analizi gereksinimlerini karşılayan çalışan bir sistemdir.

11. TESLİM EDİLEN ÇIKTILAR

11.1. Kaynak Kod

Teslimde bulunan temel kaynak kod dosyaları:

- client.py: UDP istemci kodu
- server.py: UDP sunucu kodu
- protocol.py: Paket formatı ve SHA-256 yardımcıları
- logger.py: CSV/JSON loglama
- analyzer.py: Analiz, grafik ve teknik yorum üretimi
- experiments.py: Otomatik deney senaryoları
- tcp_client.py, tcp_server.py: TCP karşılaştırması için yardımcı betikler
- tests/: Unit testler

11.2. README

README dosyasında projenin amacı, kurulum adımları, server/client/analyzer/experiments komutları, unit test komutu, klasör yapısı ve çıktıların anlamı açıklanmıştır.

11.3. Loglar ve Sonuçlar

Örnek çıktı dosyaları:

- logs/sample_client_log.csv
- logs/sample_server_log.csv
- results/analysis_summary.csv
- results/analysis_summary.json
- results/experiment_results.csv
- results/experiment_results.json
- results/technical_interpretation.txt
- results/packet_size_results.png
- results/timeout_results.png
- results/loss_rate_results.png

11.4. Test Dosyaları

- test_files/small.txt
- test_files/medium.bin
- test_files/large.bin

11.5. Doğrulama Komutları

Final durumda aşağıdaki kontroller çalıştırılmıştır:

```
cmd
python -m compileall .
python -m unittest discover tests
python server.py --host 127.0.0.1 --port 5005 --once
python client.py --server-ip 127.0.0.1 --server-port 5005 --file test_files/medium.bin
python analyzer.py --log logs/sample_client_log.csv
python experiments.py
```

Doğrulama sonuçları:

- Kodlar derlenmiştir.
- 7 unit test başarıyla geçmiştir.
- medium.bin aktarımı başarıyla tamamlanmıştır.
- Sunucu tarafında received_files/medium.bin oluşmuştur.
- Orijinal ve alınan dosyanın SHA-256 hash değerleri eşleşmiştir.
- Analyzer çıktısında integrity_ok=true görülmüştür.
- Deneyler çalışmış ve grafikler oluşmuştur.

12. KAYNAKLAR VE KULLANILAN KÜTÜPHANELER

Bu projede hazır dosya aktarım kütüphanesi kullanılmamıştır. Dosya aktarımı Python socket modülü ile raw UDP socket programming kullanılarak yapılmıştır.

Standart Python modülleri:

- socket
- time
- hashlib
- struct
- argparse
- csv
- json
- random
- os
- subprocess
- unittest

Harici kütüphaneler:

- pandas: Deney sonuçlarının okunması ve işlenmesi için kullanılmıştır.
- matplotlib: Grafik üretimi için kullanılmıştır.

Kaynaklar:

- Bilgisayar Ağları Dersi Dönem Projesi Föyü
- Python resmi dokümantasyonu: socket, hashlib, struct, csv, json, unittest
- pandas ve matplotlib resmi dokümantasyonları

EKLER

Ek A: Grafik Dosyaları

- results/packet_size_results.png
- results/timeout_results.png
- results/loss_rate_results.png

Ek B: GitHub Deposu

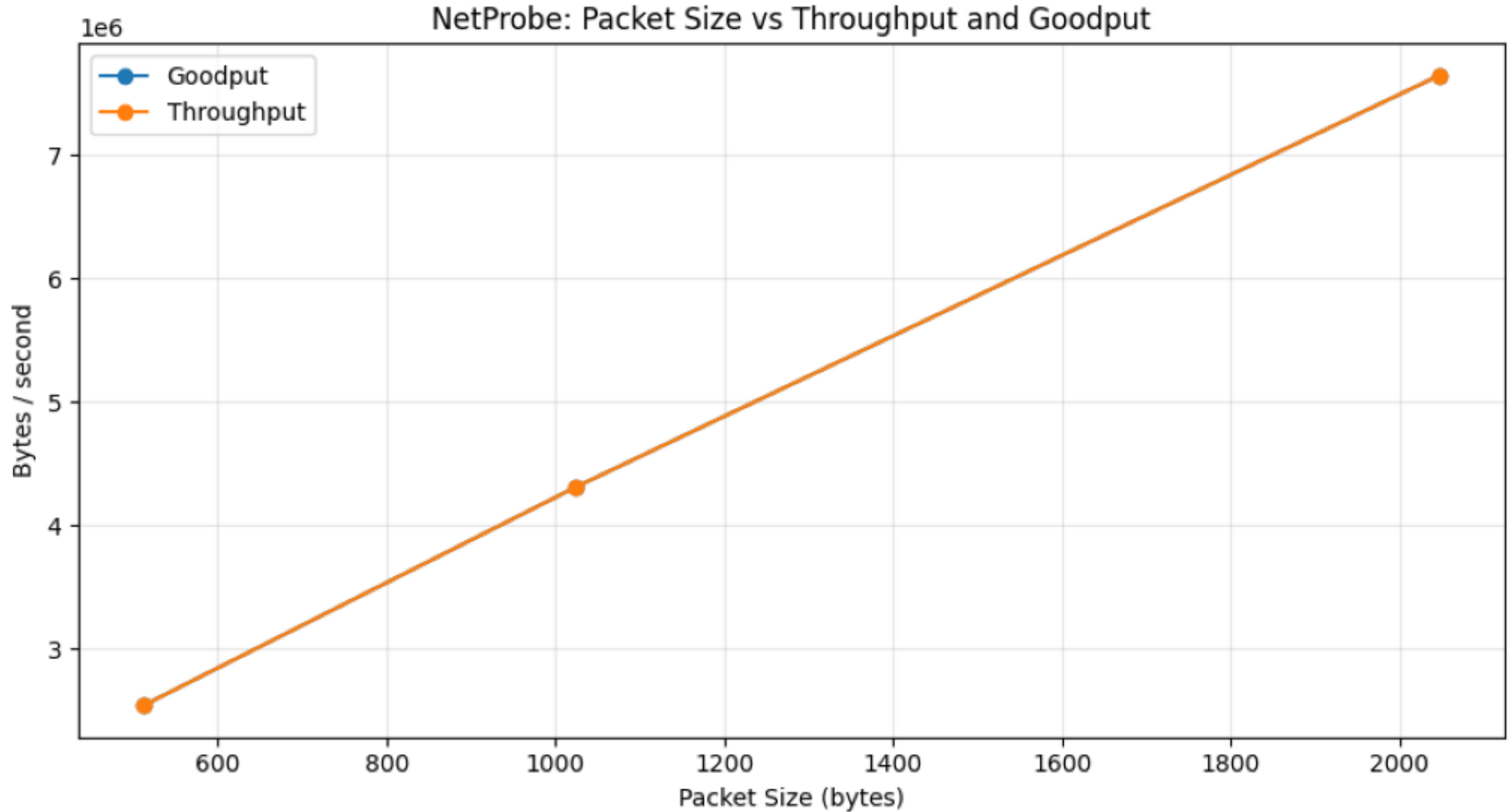
text
<https://github.com/Rima2002/netprobe>

Ek C: Örnek SHA-256 Kontrolü

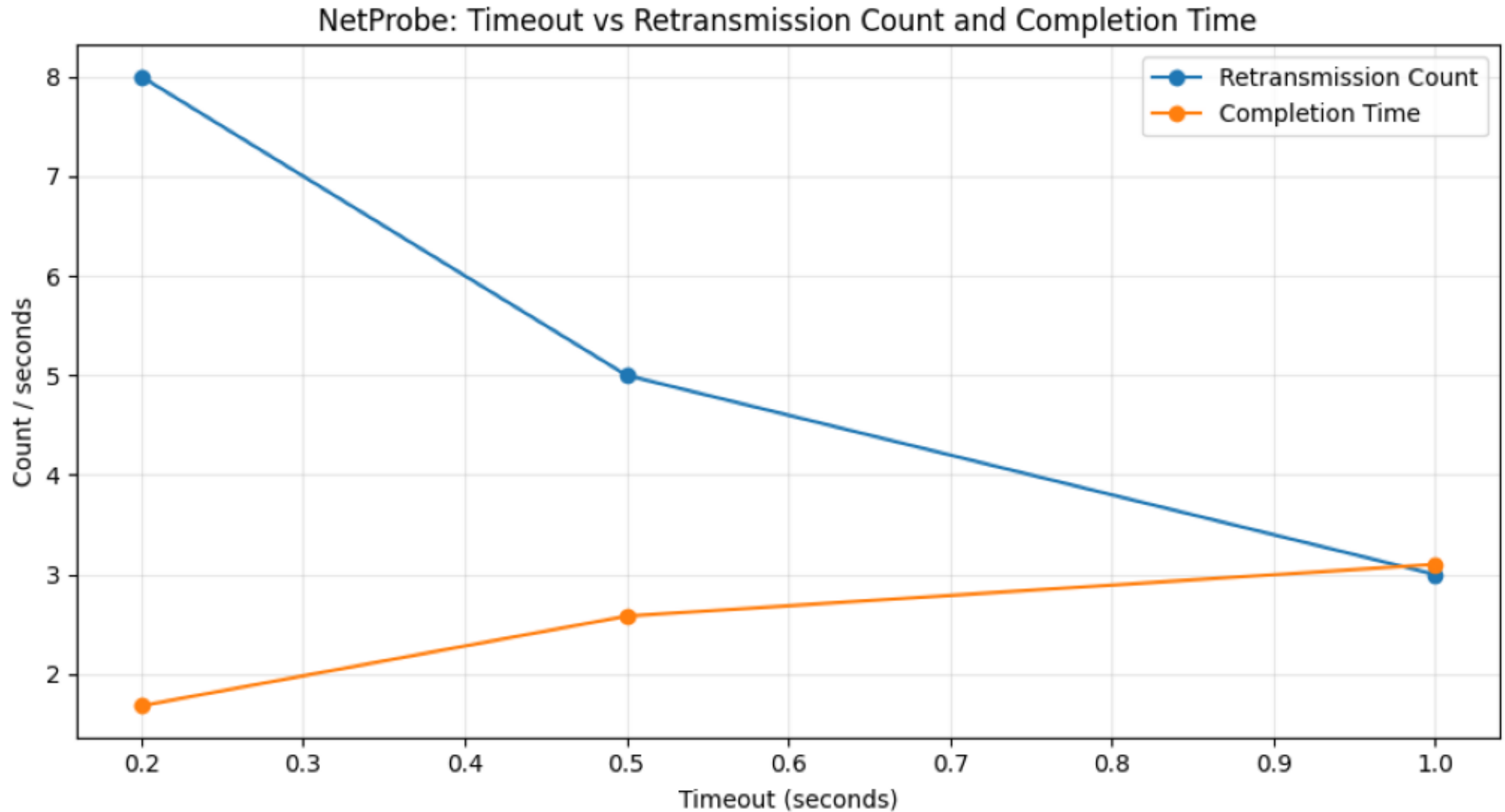
Aktarım sonrası yapılan kontrol sonucunda orijinal ve alınan dosya hash değerlerinin eşleştiği doğrulanmıştır:

text
test_files/medium.bin == received_files/medium.bin
integrity_ok = true

Ek A - Paket Boyutu Deneyi



Ek B - Timeout Deneyi



Ek C - Yapay Kay?p Oran? Deneyi

